

NOS-Gate: Queue-Aware Streaming IDS for Consumer Gateways under Timing-Controlled Evasion

Muhammad Bilal , Senior Member, IEEE, Omer Tariq  Hasan Ahmed

Abstract—Timing and burst patterns can leak through encryption, and an adaptive adversary can exploit them. This undermines metadata-only detection in a stand-alone consumer gateway. Therefore, consumer gateways need streaming intrusion detection on encrypted traffic using metadata only, under tight CPU and latency budgets. We present a streaming IDS for stand-alone gateways that instantiates a lightweight two-state unit derived from Network-Optimised Spiking (NOS) dynamics [1] per flow, named *NOS-Gate*. NOS-Gate scores fixed-length windows of metadata features and, under a K -of- M persistence rule, triggers a reversible mitigation that temporarily reduces the flow's weight under weighted fair queueing (WFQ). We evaluate NOS-Gate under timing-controlled evasion using an executable *worlds* benchmark that specifies benign device processes, auditable attacker budgets, contention structure, and packet-level WFQ replay to quantify queue impact. All methods are calibrated label-free via burn-in quantile thresholding. Across multiple reproducible worlds and malicious episodes, at an achieved 0.1% false-positive operating point, NOS-Gate attains 0.952 incident recall versus 0.857 for the best baseline in these runs. Under gating, it reduces p99.9 queueing delay and p99.9 collateral delay with a mean scoring cost of $\approx 2.09 \mu\text{s}$ per flow-window on CPU.

Index Terms—consumer gateway, metadata-only intrusion detection, timing-controlled evasion, network-optimised spiking (NOS), weighted fair queueing

I. INTRODUCTION

Home gateways and consumer hubs sit between consumer devices and encrypted services (web and cloud). Encryption limits payload inspection at the edge, but it does not remove traffic side channels such as timing, packet sizes, and burst structure [2], [3]. These signals have been used for device identification and security monitoring [4]–[7], but the gateway setting imposes two constraints that shape what is practical. First, detection must run continuously with low latency and modest CPU, which makes large models and heavy training difficult to justify [8], [9]. Second, an attacker can often change packet timing without changing endpoints, ports, or encrypted content.

Timing control is therefore a realistic evasion surface. Prior work shows that timing and burst patterns can still leak structure under encryption [3]. The same idea can be used offensively: an attacker can reshape inter-arrival times and bursts to resemble

benign device rhythms while still pursuing a hidden attack objective [10], [11]. This raises an operational IDS question that is not captured well by offline accuracy alone: can a gateway detect quickly enough to act, at a low false-alarm rate, using only metadata, against timing-shaped adversaries?

We address this question with *NOS-Gate*, a streaming IDS for stand-alone gateways derived from Network-Optimised Spiking (NOS) dynamics [1], [12]. NOS-Gate instantiates a lightweight two-state dynamical unit per flow. An excitability-like state accumulates evidence from normalised metadata deviations, while a recovery-like state provides short-term memory and suppresses repeated triggering. The detector emits an event-like surrogate and applies a persistence rule to stabilise decisions. This design is deliberately simple: compact state updates fit gateway budgets while still expressing persistence and hysteresis, which are hard to add to a simple window-based score with no memory. [13], [14].

Crucially, we connect detection to a gateway-safe action. When alarms persist, NOS-Gate temporarily reduces the weight of the flagged flow under weighted fair queueing (WFQ). This aims to reduce queue-tail harm while limiting collateral impact on benign traffic [15]–[18]. Figure 1 summarises the online loop (windowing, scoring, persistence, and WFQ gating) and the offline evaluation harness used to measure queue impact.

Recent encrypted-traffic security systems have improved robustness and deployment realism, but they usually make different assumptions than a consumer gateway IDS. HyperVision detects unknown encrypted malicious traffic using flow-interaction graphs [19]. RAPIER studies detection under low-quality training data [20]. Wedjat targets evasive behaviour using real-time causal analysis [21]. ET-SSL explores self-supervised contrastive learning for encrypted-traffic anomaly detection [22]. These works reinforce that encrypted side channels remain useful, but they typically do not evaluate the full gateway action loop (detection $\rightarrow$ mitigation $\rightarrow$ queue impact) under an explicit timing-control budget, and they do not standardise a stand-alone label-free calibration protocol. NOS-Gate is built around those gateway requirements. Table IV provides an explicit mapping from NOS symbols to their IDS interpretation in NOS-Gate, so readers can audit what is reused unchanged and what is reinterpreted for detection and action.

A second contribution is evaluation methodology. Public IoT corpora and botnet traces are valuable, but they often provide limited coverage of timing-controlled evasion and rarely offer an auditable link from attacker constraints to queue-level harm at the gateway [23]–[25]. We therefore introduce an

Muhammad Bilal and Hasan Ahmed are with the School of Computing and Communications, Lancaster University, LA1 4WA, Lancaster, U.K. E-mail: m.bilal@ieee.org, h.ahmed@lancaster.ac.uk.

Omer Tariq is with the School of Computing, Korea Advanced Institute of Science and Technology, Daejeon, 34141, South Korea. E-mail: omeritariq@kaist.ac.kr

executable *worlds* benchmark that specifies benign device processes and contention structure, constrains adversaries by auditable budgets, and uses packet-level WFQ replay to quantify user-visible impact. We evaluate all detectors under the same stand-alone, label-free protocol: an assumed-benign burn-in segment, per-flow thresholds set by high quantiles of burn-in scores, and labels used only for reporting on held-out time [8], [9], [26].

Encrypted-traffic IDS work often improves detection performance under a dataset protocol, but at least one gateway-facing element is commonly left implicit: (i) a stand-alone, label-free protocol that fixes an explicit false-alarm budget without supervised tuning, (ii) an explicit timing-control threat model with auditable operational budgets, and (iii) an action-impact evaluation that measures whether mitigation improves user-visible queue outcomes rather than only producing alarms. This paper adds all three in a single executable loop: NOS-Gate as a compact two-state streaming detector calibrated without labels, *worlds* as a reproducible benchmark with auditable timing and contention budgets (including feasibility reporting), and packet-level WFQ replay to quantify queue-tail and collateral impact under reversible gating.

We make three contributions that together support a practical, auditable gateway IDS under timing-controlled evasion:

- We propose a two-state detector *NOS-Gate*, a streaming, metadata-only gateway IDS with a direct action hook: a per-flow two-state detector with persistence-based alerting that drives WFQ deprioritisation.
- We introduce a reproducible *worlds* benchmark with published contention structure, auditable attacker budgets, and packet-level WFQ replay, enabling queue-tail and collateral evaluation under timing-shaped attacks.
- We standardise a stand-alone operating protocol at explicit false-alarm budgets using burn-in quantile thresholding and persistence, and we report achieved false-positive rates, incident recall, and time-to-detect under the same calibration rule.

We do not claim payload inspection, DPI deployment in ISP-grade firmware, or that our synthetic worlds reproduce any single public dataset. Instead, under the published feature contract and attacker budgets, NOS-Gate provides a stronger detection-and-action trade-off than the evaluated label-free baselines: higher incident detection at the same achieved false-alarm budget, and improved queue-tail outcomes when WFQ gating is enabled. This work do not claim payload inspection, DPI deployment in ISP-grade firmware, or that synthetic worlds reproduce any single public dataset. We present the benchmark as a falsifiable protocol: new worlds, budgets, and contention structures can be added to test generality and failure modes. Table I groups representative encrypted-traffic systems by operating model and evaluation axes that matter for consumer gateway comparability.

II. SYSTEM MODEL (CONSUMER GATEWAY)

We model a stand-alone consumer gateway that observes packet metadata behind NAT and produces streaming decisions

at a fixed cadence. Time is divided into non-overlapping windows of length $\Delta t = 250$ ms (Section III-A). The detector operates on directed five-tuple flows and is calibrated without labels using an assumed-benign burn-in segment. Flows may start and stop during a horizon. For a newly observed flow, we compute scores immediately, but we enable thresholded alarming only after the burn-in segment needed for quantile calibration has accrued; before that, we suppress alarms and do not allow WFQ gating.

Inputs and operating protocol.: The feature stream is limited to fields visible at the gateway: packet headers, timestamps, and sizes, plus DNS and TLS handshake metadata when present (for example, SNI and certificate validity days). We do not use payload bytes, deep packet inspection features, or host logs. If DNS/TLS fields are absent, NOS-Gate falls back to timing and size features without changing the model. To avoid mixing feature dimensions within a run, we evaluate a fixed feature contract per run (for example, timing+contention only, or timing plus DNS/TLS when uniformly available). The run manifest records the feature contract used.

For each flow i and window t , the detector outputs a score $s_{i,t}$ and an alarm $a_{i,t} = \mathbb{I}\{s_{i,t} \geq \vartheta_i\}$.

Thresholds are calibrated label-free from the burn-in segment. Let the horizon be H windows and let the burn-in length be $H_{\text{burn}} = \lfloor 0.6H \rfloor$. In our worlds, $H = 57,600$ and $H_{\text{burn}} = 34,560$ windows at $\Delta t = 250$ ms (about 2.4 hours). We set

$$\vartheta_i = \text{Quantile}_q\left(\{s_{i,t} : t = 0, \dots, H_{\text{burn}} - 1\}\right), \quad (1)$$

with $q \in \{0.99, 0.999\}$ as the operating knob (we report both). Before ϑ_i is available, we compute $s_{i,t}$ causally but suppress alarms ($a_{i,t} = 0$) and do not trigger gating; after burn-in, thresholds are frozen and performance is reported on the held-out segment, including achieved benign false-positive rate.

To avoid flapping, we apply a K -of- M persistence rule. A flow becomes actionable only if at least K alarms occur in the most recent M windows (defaults $K = 3$, $M = 8$):

$$z_{i,t} = \mathbb{I}\left\{\sum_{\tau=t-M+1}^t a_{i,\tau} \geq K\right\}, \quad (2)$$

where the actionable indicator $z_{i,t}$ is defined by the rolling K -of- M rule above. WFQ gating is implemented as a reversible quarantine timer: when $z_{i,t} = 1$ for a flow, we apply $\omega_i = \omega_-$ for a fixed quarantine duration T_g . If further actionable triggers occur while a flow is already gated, the quarantine interval is extended accordingly. If no further triggers occur, the flow returns automatically to ω_0 when the quarantine timer expires.

Queueing model and WFQ action.: We evaluate action impact through packet-level queue replay at the gateway. Flows that share the uplink enter a work-conserving weighted fair queueing (WFQ) scheduler. Let $\omega_i(t)$ be the weight assigned to flow i at time t . Under normal operation, $\omega_i(t) = \omega_0$. Under gating, a flow that is actionable ($z_{i,t} = 1$) is deprioritised by setting $\omega_i(t) = \omega_-$ with $\omega_- < \omega_0$ for a quarantine duration T_g (or until the flag clears, whichever is longer). We report both detection metrics and queue-tail metrics, including collateral delay experienced by benign flows. WFQ gating is reversible and avoids immediate blocking while still limiting queue-tail harm when alarms persist (Figure 1).

TABLE I
COMPARISON OF ENCRYPTED-TRAFFIC SECURITY SYSTEMS FOR STAND-ALONE GATEWAY DEPLOYMENT

Group / work	Inputs	Training & calibration	Online	Robustness / evasion	Action / impact	Fit to assumptions	NOS-Gate
<i>G0. Gateway action loop (detection → mitigation → queue impact)</i>							
NOS-Gate (this paper)	Per-flow metadata + contention signals	Label-free burn-in; per-flow high-quantile thresholds; K -of- M persistence	Yes	Explicit timing-control budgets; feasibility reported	WFQ weight gating; queue-tail & collateral via replay	Direct match: stand-alone calibration and action-impact loop	
<i>G1. Interaction-aware encrypted-traffic detection (detection-first)</i>							
HyperVision [19]	Encrypted traffic features + flow-interaction graph	Dataset-trained; calibration follows paper/dataset protocol	Yes	Targets unknown/novel threats (not posed as timing-budget shaping)	No mitigation or queue impact	Closest on online inference, but different calibration and threat model	
<i>G2. Robust learning under weak supervision / data quality shift</i>							
RAPIER [20]	Encrypted traffic detection features	Robust training under low-quality data; typically offline	Often	Robustness to data quality shift (rather than timing budgets)	No action loop	Useful robustness angle, but not an action-coupled gateway protocol	
<i>G3. Evasion emphasis (action-free evaluation)</i>							
Wedjat [21]	Encrypted traffic signals + causal analysis components	Dataset-driven learning and evaluation protocol	Yes	Explicit evasion emphasis; real-time framing	No mitigation or queue impact	Threat focus is closer, but lacks stand-alone calibration and impact loop	
<i>G4. Self-supervised representation learning (pretrain then downstream scoring)</i>							
ET-SSL [22]	Encrypted traffic features	Self-supervised pretraining; downstream calibration varies by use case	Often	Not formulated as timing-budget shaping; robustness depends on downstream task	No action loop	Promising feature learning, but comparability hinges on calibration and impact evaluation	

Runtime measurability and clique approximation.: In *worlds*, contention cliques and weights W are specified by construction to make interference and $\rho(W)$ auditable. In a consumer gateway, clique membership is not given as an explicit graph label, but the relevant observables are available at the scheduler boundary: per-queue backlog/occupancy (or delay estimates), per-class service counters, and WFQ configuration/state (weights and per-class rate share). Operationally, we approximate a “clique” as the set of flows that share the same constrained scheduling domain, such as a WAN egress queue and traffic class. Per-flow membership is obtained from the gateway’s own flow-to-class mapping (five-tuple classification or policy rules). This is sufficient for the contention features we use (share , q) and for implementing reversible weight gating without requiring an online oracle for W .

A. Worlds benchmark: contract and outputs

The primary evidence in this paper is produced by a reproducible synthetic benchmark called *worlds*. A *world* is a fully specified, executable instance that fixes benign device processes, malicious episode generators, contention structure, and an evaluation harness for queue impact. Each world publishes: (i) benign device classes and traffic generators, (ii) malicious episode generators, (iii) contention structure (cliques and weights W , including a target band for $\rho(W)$), (iv) a packet trace with flow keys, (v) the derived windowed feature stream, (vi) labels used only for reporting, and (vii) WFQ replay configuration and queue summaries. The released artefacts include world ids, random seeds, configuration hashes, and manifests linking raw and derived outputs.

Budgets, feasibility, and timeline.: Timing-controlled evasion is defined by auditable operational budgets ($R_i^{\min}$, ε , δ_q) and a feasibility protocol. We define these budgets in the

TABLE II
SCALE OF THE *worlds* BENCHMARK USED IN THIS PAPER. OFFERED-LOAD STATISTICS ARE DERIVED FROM BENIGN GENERATOR PARAMETERS (MEAN IAT FAMILY PARAMETERS AND MEAN PACKET SIZES); REALISED TRACES VARY STOCHASTICALLY AROUND THESE MEANS.

Quantity	Value (this paper)
Window length Δt	250 ms
Horizon H	57,600 windows (4 hours)
Devices / directed flows	32 devices; 64 directed flows (2 per device)
Malicious flows / episodes	3 malicious flows per world (active at any given time)
Clique structure	4 cliques, size 16 each
Contention density	0.25 (world construction setting)
$\rho(W)$ band (achieved)	≈ 0.90 per clique (by construction; narrow band around target)
Per-flow rate (pkts/s)	min / median / max $\approx 0.16/4.68/14.15$
Per-flow offered load (kb/s)	min / median / max $\approx 0.61/15.32/60.85$
Total offered load (Mb/s)	≈ 1.59 (sum over 64 flows)

threat model (Section III-D, Table III) and report feasibility rates for each budget setting. Within each world, time is split chronologically into burn-in, validation, and test. During burn-in we update the online normaliser, train label-free baselines, and collect burn-in scores to set per-flow thresholds ϑ_i . Validation is used only for design-time selection from a small fixed hyperparameter grid. Test uses frozen detectors and frozen ϑ_i to report achieved false-positive rate, recall, detection delay, and queue-tail outcomes with and without gating.

III. THREAT MODEL

We consider a stand-alone consumer gateway that observes packet metadata behind NAT and emits streaming, flow-level decisions at a fixed window cadence. The attacker can reshape packet timing and, within bounds, packet sizes. Evasion is

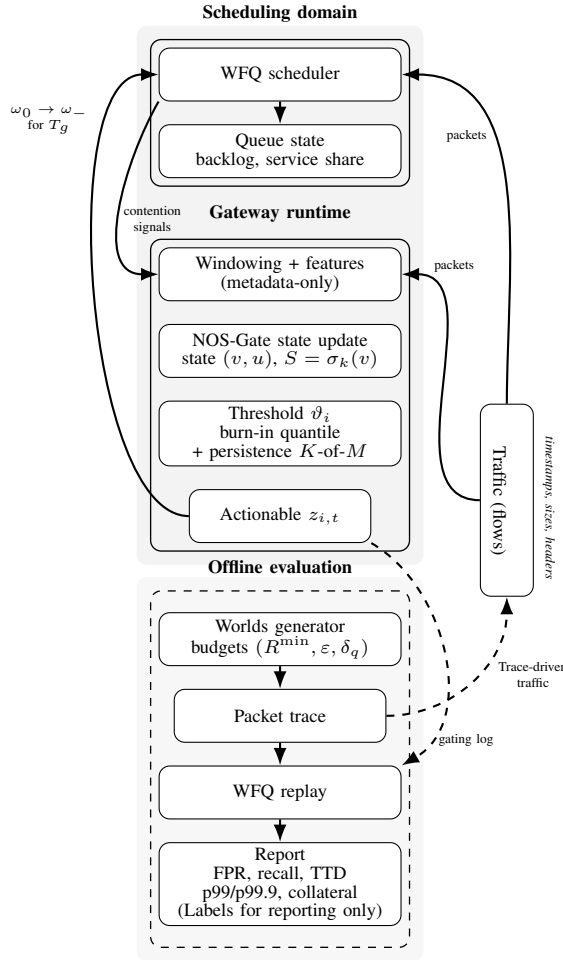


Fig. 1. Online, the gateway windows metadata-only traffic, updates NOS-Gate state (v, u) , applies burn-in quantile thresholding and a K -of- M persistence rule to produce an actionable flag $z_{i,t}$, and temporarily reduces WFQ weight $\omega_0 \rightarrow \omega_-$ for T_g . Offline, *worlds* generates trace-driven traffic and labels, runs the same runtime to log gating decisions, and replays WFQ to report detection metrics and queue-tail and collateral outcomes; labels are used only for reporting. Solid arrows: deployed online loop. Dashed arrows: offline evaluation harness.

constrained by explicit operational budgets that make attacks auditable in our *worlds* benchmark.

A. Observation model and feature access

Packets are indexed by n , with arrival times $t_n \in \mathbb{R}_{\geq 0}$ and lengths $\ell_n \in [\ell_{\min}, \ell_{\max}]$. A directed flow i is the five-tuple (srcIP, dstIP, srcPort, dstPort, proto). Time is partitioned into non-overlapping windows of length Δt , indexed by $t \in \{0, 1, \dots, H-1\}$.

Let $N_{i,t}$ be the number of packets from flow i in window t ,
$$N_{i,t} = |\{n \in i : t_n \in [t\Delta t, (t+1)\Delta t)\}|. \quad (3)$$
 When $N_{i,t} \geq 2$, define the inter-arrival-time (IAT) multiset within the window as
$$\mathcal{I}_{i,t} = \{t_{n+1} - t_n : n, n+1 \in i, t_n, t_{n+1} \in [t\Delta t, (t+1)\Delta t)\}. \quad (4)$$

The detector receives a feature vector $x_{i,t} \in \mathbb{R}^d$ computed causally from packet timestamps and sizes, optional DNS/TLS handshake metadata when visible, and contention signals

available at the gateway (Section V). No payload content and no host logs are used.

B. Benign traffic and malicious episodes

Within a world, each flow i is generated by either a benign process or a malicious episode generator. Let $\mathbb{P}_i^{\text{ben}}$ and $\mathbb{P}_i^{\text{mal}}$ denote the per-window distributions of $x_{i,t}$ in benign and malicious modes. Flows that share the same uplink scheduling domain are grouped into contention cliques. We represent clique-level interference using a nonnegative weight matrix $W = [w_{ij}]$, and we report its spectral radius $\rho(W)$ as a compact summary of contention intensity.

C. Adversary capabilities and knowledge

We assume a remote adversary that can compromise IoT devices and generate malicious traffic while remaining behind NAT. The adversary can: (i) use encrypted channels, (ii) choose protocols and endpoints consistent with common device behaviour, and (iii) reshape packet timing and, within bounds, packet sizes. The adversary knows the feature contract and the window length Δt .

The defender calibrates thresholds in a label-free, stand-alone way during an assumed-benign burn-in segment (Section II). This reflects a standard operational assumption: burn-in is not dominated by malicious behaviour.

D. Operational budgets for timing-controlled evasion

Evasion is constrained by three budgets that capture objective, timing stealth, and contention stealth. For a flow i , the budgets are $(R_i^{\min}, \epsilon, \delta_q)$ and are enforced by the worlds generator (Table III).

For timing stealth, each malicious flow i is paired with a benign IAT reference distribution $P_i^{\text{ben,iat}}$ constructed within the same world, using the benign process that matches i 's device class and endpoint template. This makes the timing constraint auditable per world. For readability, we use $P_i^{\text{ben,iat}}$ to denote the (fixed) benign IAT reference distribution for flow i , and $\hat{P}_{i,t}^{\text{iat}}$ for the empirical IAT distribution observed within window t ; this notation is used consistently throughout the threat model and Algorithm 1.

IAT-based distortion is undefined when $N_{i,t} < 2$. We therefore apply the timing distortion constraint only on windows that contain at least two packets:

$$\mathcal{T}_i = \{t \in \{0, \dots, H-1\} : N_{i,t} \geq 2\}. \quad (5)$$

This does not remove constraints in sparse windows. The detector still observes packet counts, rates, and size-based features in every window. The restriction applies only to the IAT distortion term because it is undefined otherwise.

For contention stealth, let $C(i)$ be the contention clique containing flow i . Under baseline WFQ replay *without defensive gating*, let d_n be the queueing delay (enqueue-to-dequeue time) experienced by packet n in clique C . Define the clique mean delay over the horizon as

$$\bar{D}_C = \frac{1}{|\mathcal{P}_C|} \sum_{n \in \mathcal{P}_C} d_n, \quad (6)$$

TABLE III
OPERATIONAL BUDGETS DEFINING TIMING-CONTROLLED EVASION IN worlds.

Budget	Symbol	Units	Definition
Throughput floor	$R_i^{\min}$	bytes	$\sum_{n \in i, t_n \in [0, H\Delta t)} \ell_n \geq R_i^{\min}$
Timing distortion	ε	s	$\frac{1}{ \mathcal{T}_i } \sum_{t \in \mathcal{T}_i} W_1(\hat{P}_{i,t}^{\text{iat}}, P_i^{\text{ben,iat}}) \leq \varepsilon$
Contention stealth	δ_q	s	$\bar{D}_{C(i)}^{\text{atk}} - \bar{D}_{C(i)}^{\text{ben}} \leq \delta_q$

where $\mathcal{P}_C$ is the set of packets belonging to flows in clique C during $[0, H\Delta t)$. We write $\bar{D}_C^{\text{ben}}$ and $\bar{D}_C^{\text{atk}}$ for the mean delays under benign-only and attack-injected traces, respectively.

Some triples $(R_i^{\min}, \varepsilon, \delta_q)$ are infeasible for a given world and clique load, so feasibility is treated as a first-class outcome. When a requested triple is infeasible, the generator either declares infeasibility under the requested budgets or attempts to restore feasibility by reducing offered load while preserving the throughput floor when possible. If the throughput floor cannot be met jointly with (ε, δ_q) , the attempt is declared infeasible. We report feasibility rates for each budget setting.

E. Defender evaluation under stand-alone thresholding

For flow i and window t , the detector outputs a score $s_{i,t}$ and a binary alarm

$$a_{i,t} = \mathbb{I}\{s_{i,t} \geq \vartheta_i\}, \quad (7)$$

where ϑ_i is set by high-quantile burn-in thresholding (Eq. (1)). Let $\mathcal{A}(R_i^{\min}, \varepsilon, \delta_q)$ denote the admissible adversarial policies that satisfy Table III. For a fixed quantile q , we evaluate robustness through the achieved detection probability under admissible attacks,

$$\min_{P \in \mathcal{A}(R_i^{\min}, \varepsilon, \delta_q)} \Pr[s(x) \geq \vartheta_i \mid \text{mal}], \quad (8)$$

and we report the achieved benign false-positive rate induced by the same ϑ_i on the held-out test segment.

F. Evasive policy class: projection-based C2

To avoid cherry-picked evasions, we use a fixed, auditable policy class that directly enforces the three budgets. The generator starts from a malicious proposal schedule, then (i) projects timing within each window to satisfy the Wasserstein constraint, (ii) repairs sizes to maintain the throughput floor, and (iii) enforces contention stealth through iterative load reduction with baseline WFQ replay. To satisfy the three budgets, we apply the projection-based procedure summarized in Algorithm 1. The term *budget-projection* refers to taking an initial malicious packet schedule and projecting it into the feasible set defined by $(R_i^{\min}, \varepsilon, \delta_q)$ via an order-preserving time warp (quantile mapping of within-window IATs) plus local size repair to maintain the throughput floor. Here “projection” is used in the optimisation sense: starting from a proposed malicious schedule, we apply an order-preserving time-warp within each window (implemented as quantile mapping of sorted IATs with minimal interpolation) to satisfy the Wasserstein radius constraint, then repair sizes to maintain the throughput floor and iterate load reduction to satisfy contention stealth.

Algorithm 1 Budget-projection evasive C2 generator (per flow)

Require: Benign IAT reference $P_i^{\text{ben,iat}}$, size bounds $[\ell_{\min}, \ell_{\max}]$, window size Δt , horizon H , bytes floor $R_i^{\min}$, distortion radius ε , delay tolerance δ_q , max iters $I_{\max}$
Ensure: Packet times $\{t_n\}$ and sizes $\{\ell_n\}$ for flow i over $[0, H\Delta t)$, or infeasibility flag

- 1: **Initialise** a proposal schedule with packet times and sizes satisfying $\sum_n \ell_n \geq R_i^{\min}$
- 2: **for** $t = 0$ to $H - 1$ **do**
- 3: Extract packets in window t and compute $N_{i,t}$
- 4: **if** $N_{i,t} \geq 2$ **then**
- 5: Compute $\hat{P}_{i,t}^{\text{iat}}$ and $d_t \leftarrow W_1(\hat{P}_{i,t}^{\text{iat}}, P_i^{\text{ben,iat}})$
- 6: **if** $d_t > \varepsilon$ **then**
- 7: **Project timing in window:** apply an order-preserving time-warp on $[t\Delta t, (t+1)\Delta t)$ so the resulting IAT empirical distribution satisfies $W_1 \leq \varepsilon$ (implementation: quantile mapping on sorted IATs with minimal interpolation)
- 8: **end if**
- 9: **end if**
- 10: **Repair throughput locally:** adjust packet sizes within $[\ell_{\min}, \ell_{\max}]$ to keep cumulative bytes on track for $R_i^{\min}$
- 11: **end for**
- 12: Run baseline WFQ replay (no gating) for clique $C(i)$ to estimate $\bar{D}_{C(i)}^{\text{atk}}$
- 13: **for** $\ell = 1$ to $I_{\max}$ **do**
- 14: **if** $\bar{D}_{C(i)}^{\text{atk}} - \bar{D}_{C(i)}^{\text{ben}} \leq \delta_q$ **then**
- 15: **return** packet times and sizes
- 16: **end if**
- 17: **Reduce offered load:** decrease packet emission while attempting to preserve $\sum_n \ell_n \geq R_i^{\min}$
- 18: Re-apply the per-window projection and size repair steps
- 19: Re-estimate $\bar{D}_{C(i)}^{\text{atk}}$ by baseline WFQ replay
- 20: **end for**
- 21: **return infeasible** under $(R_i^{\min}, \varepsilon, \delta_q)$

G. Optional protocol side-channel constraints

When DNS or TLS handshake metadata is available, we include the DNS/TLS features listed in Table V as part of the fixed contract for that run. When such metadata is not present for a particular flow-window, the corresponding DNS/TLS feature values default to zero under the fixed contract. We also report results under a timing+contention-only contract (i.e., without DNS/TLS features) when those fields are not available in a uniform way. The run manifest records the feature contract and discrete settings (including B and the z-score clip magnitude).

IV. NOS-GATE DETECTOR: MAPPING IDS TO NOS DYNAMICS

NOS-Gate instantiates the NOS two-state unit as a streaming intrusion detector at a consumer gateway. The two-state dynamics, bounded excitability, recovery coupling, and damping terms are inherited from NOS. The change is

the meaning of the exogenous drive: in NOS networking it represents queue and interference forcing, whereas in NOS-Gate it is a metadata-only evidence signal computed per flow per window. This retains the NOS structure while changing what the “input” represents.

Table IV provides an auditable correspondence between the NOS symbols and their IDS interpretation, and makes clear what is reused unchanged and what is reinterpreted for detection and action.

A. Design rationale: why a two-state NOS unit helps under timing control

NOS-Gate targets the gateway loop in Figure 1: flows must be scored online at a fixed cadence, thresholds must be set without labels, and decisions should drive a reversible action with measurable queue impact. Under the threat model, the attacker can time-warp within windows and reshape burst structure subject to a throughput floor and a timing-distortion budget. This often reduces sharp per-window signatures and pushes attacks toward smaller deviations that persist over longer periods.

Stand-alone thresholding makes this harder. Each flow threshold ϑ_i is set by a high burn-in quantile (Eq. (1)) and then frozen. A detector that relies on rare, high-amplitude score spikes is easier to suppress because the attacker only needs to keep scores below a tail threshold. A detector that can accumulate modest deviations over time is more likely to cross a high quantile without requiring any single window to look extreme.

With coupling disabled by default ($g = 0$), NOS-Gate acts as a per-flow streaming mechanism driven by the scalar evidence $E_{i,t}$ in Eq. (10). Around the benign operating band, the v -update behaves like a leaky evidence accumulator, while the recovery state u provides slower negative feedback. The leak terms prevent unbounded accumulation, the bounded nonlinearity reduces sensitivity to rare transients, and the explicit $-u$ term suppresses repeated triggering after sustained elevation. The event surrogate $S_{i,t} = \sigma_k(v_{i,t})$ converts this filtered state into a bounded readout that is straightforward to calibrate using a high-quantile rule.

B. From gateway flows to NOS nodes

We model each active directed flow i as a node in a contention structure with nonnegative weights $W = [w_{ij}]$. Two flows are neighbours if they compete for the same gateway scheduling domain (uplink WFQ domain or a contention clique). The NOS graph-local input is retained:

$$I_{i,t} = g \sum_{j \in \mathcal{N}(i)} w_{ij} S_{j,t-L_{ij}}, \quad (9)$$

where g is a global coupling gain, $L_{ij} \in \mathbb{Z}_{\geq 0}$ is an integer delay in windows, and $S_{j,t}$ is an event surrogate emitted by flow j (defined below). In the main results we set $g = 0$ unless stated otherwise; coupling is evaluated in ablations. We update all flows synchronously once per window. Since the neighbour surrogate $S_{j,t}$ is emitted at window t and is only available when advancing to $t+1$, the smallest realised coupling lag is one full window. We therefore parameterise an *additional*

delay $\tau_{ij} \in \mathbb{Z}_{\geq 0}$ beyond this inevitable pipeline delay and define $L_{ij} = 1 + \tau_{ij}$, so $\tau_{ij} = 0$ corresponds to the minimum one-window lag.

C. State interpretation for IDS

NOS-Gate reuses the two NOS states with an IDS interpretation that preserves their mathematical roles:

$v \mapsto$ bounded evidence accumulation (suspicion),

$u \mapsto$ recovery state (suppression resource).

Operationally, v behaves like a virtual evidence queue that rises when metadata features deviate from benign patterns and drains under damping. The recovery state u increases under sustained elevation and pulls v down via the explicit $-u$ term, which reduces rapid re-triggering after a burst of alarms. When we include u in a score readout, we treat it as a persistence indicator correlated with sustained deviation, not as a direct alarm trigger by itself.

D. Evidence drive as a magnitude input

For each flow i and window t , we compute a normalised feature vector $\hat{x}_{i,t} \in \mathbb{R}^d$ (Section V) and map it to a scalar evidence drive

$$E_{i,t} = \zeta \|\hat{x}_{i,t}\|_p, \quad (10)$$

where $p \geq 1$ and $\zeta > 0$ are hyperparameters. We treat $E_{i,t}$ as constant over the window $[t\Delta t, (t+1)\Delta t)$. Because $\hat{x}_{i,t}$ is produced by online z-scoring, the norm aggregates multi-feature deviations without labels and avoids learning per-feature weights.

E. NOS dynamics for IDS (windowed discretisation)

NOS-Gate runs once per window, so we discretise the NOS two-state dynamics with an explicit Euler step of size Δt . Let $v_{i,t} \approx v_i(t\Delta t)$ and $u_{i,t} \approx u_i(t\Delta t)$.

Event surrogate (readout used for coupling and scoring).

$$S_{i,t} = \sigma_k(v_{i,t} - \theta) = \frac{1}{1 + \exp(-k(v_{i,t} - \theta))}. \quad (11)$$

Neighbour drive (optional). When coupling is enabled, the NOS graph-local term is Eq. (9). We update all flows synchronously once per window. Since $S_{j,t}$ is produced at window t and is only available when advancing to $t+1$, the minimum realised coupling lag is one window. We therefore parameterise an additional delay $\tau_{ij} \in \mathbb{Z}_{\geq 0}$ and implement $L_{ij} = 1 + \tau_{ij}$, so $\tau_{ij} = 0$ corresponds to the minimum one-window lag.

Bounded excitability. We use the NOS bounded nonlinearity

$$f_{\text{sat}}(v) = \frac{\alpha v^2}{1 + \kappa v^2}, \quad (12)$$

with $\alpha > 0$ and $\kappa > 0$. For numerical safety, the implementation may clip v to $[0, v_{\text{max}}]$

State update (same NOS core, with IDS-specific drive).

Define the total exogenous drive as the sum of the IDS evidence and (optionally) the neighbour drive: $A_{i,t} = E_{i,t} + I_{i,t}$ (set $I_{i,t} = 0$ when coupling is disabled). The per-window updates are

$$\begin{aligned} v_{i,t+1} = & v_{i,t} + \Delta t \left(f_{\text{sat}}(v_{i,t}) + \beta v_{i,t} + \gamma - u_{i,t} + A_{i,t} \right. \\ & \left. - \lambda v_{i,t} - \chi(v_{i,t} - v_{\text{rest}}) \right) c + \xi_{i,t} - r S_{i,t}, \end{aligned} \quad (13)$$

TABLE IV
AUDITABLE MAPPING FROM NOS (MAIN MODEL) TO NOS-GATE (IDS
INSTANTIATION).

Symbol	NOS meaning	NOS-Gate (IDS) meaning
v_i	Normalised congestion proxy	Bounded evidence accumulation, optionally clipped to $[0, v_{\max}]$
u_i	Recovery / slowdown resource	Suppression state damping repeated alarms
$f_{\text{sat}}(v)$	Bounded excitability (finite buffers)	Bounded evidence growth under deviation
$\beta v_i + \gamma$	Residual slope and offset	Bias and linear drift correction
$-u_i$	Post-burst recovery drag	Refractory suppression after alarms
$-\lambda v_i$	Service-like drainage	Evidence decay toward baseline
$-\chi(v_i - v_{\text{rest}})$	Small-signal damping	Stabilisation around benign baseline
$I_i(t)$	Graph-local coupling	Contention-driven neighbour propagation (ablation)
$S_i(t)$	Presynaptic event train	Event surrogate $S_{i,t} = \sigma_k(v_{i,t} - \theta)$
(a, b, μ)	Recovery sensitivity and decay	Suppression build-up and persistence
Exogenous drive	Telemetry forcing	Total drive $A_{i,t} = E_{i,t} + I_{i,t}$ with $E_{i,t} = \zeta \ \hat{x}_{i,t}\ _p$
Reset pathway	Differentiable reset option	Optional continuous reset $-rS_{i,t}$
Operational action	Operator control	WFQ gating: alarms reduce weight ω_i

$$u_{i,t+1} = u_{i,t} + \Delta t \left(ab v_{i,t} - (a + \mu) u_{i,t} \right). \quad (14)$$

Here $\xi_{i,t}$ is an optional zero-mean noise term (set to 0 unless enabled) and v_{rest} is the benign baseline (we use $v_{\text{rest}} = 0$ unless stated). The optional term $-rS_{i,t}$ is a continuous reset pathway (set $r = 0$ when not used). This is the only structural difference from NOS: the “input” is now $A_{i,t}$, whose IDS component is $E_{i,t}$ from Eq. (10).

F. Score, alarm, and WFQ gating action

NOS-Gate converts state into an anomaly score

$$s_{i,t} = \eta_1 S_{i,t} + \eta_2 u_{i,t}. \quad (15)$$

An alarm is $a_{i,t} = \mathbb{I}\{s_{i,t} \geq \vartheta_i\}$, where ϑ_i is set per flow by burn-in quantile thresholding (Eq. (1)). We stabilise alerts with the K -of- M persistence rule (Section II). When actionable, WFQ gating assigns $\omega_i = \omega_-$ for T_g ; otherwise $\omega_i = \omega_0$.

In the main results we use $\eta_2 = 0$, so the operational score is driven by $S_{i,t}$. We use $\eta_2 > 0$ only in ablations, where $u_{i,t}$ acts as a slow-burn persistence cue. The run manifest records (η_1, η_2) .

G. Fixed points and stability

For constant drives (that is, $E_{i,t} \equiv E_i^*$ and $I_{i,t} \equiv I_i^*$), with $\xi_{i,t} = 0$ and $r = 0$, a steady state (v_i^*, u_i^*) of the underlying NOS core satisfies

$$0 = f_{\text{sat}}(v_i^*) + \beta v_i^* + \gamma - u_i^* + E_i^* + I_i^* - \lambda v_i^* - \chi(v_i^* - v_{\text{rest}}), \quad (16)$$

together with $ab v_i^* = (a + \mu) u_i^*$ from Eq. (14). (If the optional reset $r > 0$ is enabled, it adds an additional negative feedback

term proportional to $S_i^* = \sigma_k(v_i^* - \theta)$ in the discrete-time update in Eq. (13).)

For conservative calibration checks, we isolate the coupling gain. Since $\max_x \sigma'_k(x) = k/4$, the linearised coupling path in the v -update is bounded by $\Delta t g \rho(W) k/4$. In experiments we choose (g, k) , Δt , and the world-specific $\rho(W)$ band so that this bound remains below the local damping and recovery margin induced by (λ, χ) and (a, b, μ) , and the local slope of f_{sat} around the operating band. The IDS evidence $E_{i,t}$ enters only as exogenous forcing.

We emphasise that the bound $\Delta t g \rho(W) k/4$ is used as a conservative heuristic sufficient check for selecting safe coupling settings in our discrete-time implementation; it is not a formal stability guarantee for the full nonlinear, driven system under arbitrary time-varying evidence inputs.

V. METHODS

NOS-Gate operates on per-flow packet metadata at a fixed cadence. We treat each directed five-tuple flow as a stream and summarise its packets in non-overlapping windows of length $\Delta t = 250$ ms. Each window yields one feature vector $x_{i,t}$ for flow i at window index t . In synthetic *worlds*, flow keys and (when needed) device identifiers are known by construction. In the optional corpus mode, we extract the same directed five-tuple flows from PCAP or flow logs. Device identity is not required for scoring, but when a stable identifier exists (for example, a MAC address or a DHCP lease), we maintain separate normalisation statistics per device.

A. Causal windowing and online normalisation

All feature computation is causal. Features for window t use only packets with timestamps in $[t\Delta t, (t+1)\Delta t)$, and any running statistics depend only on earlier windows. After computing $x_{i,t}$, we normalise online to reduce drift and make thresholds comparable across flows.

Let $d(i)$ be the device identifier for flow i when available (otherwise a single global bucket). For each feature index k and device bucket d , we maintain exponential running mean and variance,

$$m_{t+1}^{(d,k)} = (1 - \lambda_m) m_t^{(d,k)} + \lambda_m x_t^{(d,k)}, \quad (17)$$

$$q_{t+1}^{(d,k)} = (1 - \lambda_v) q_t^{(d,k)} + \lambda_v \left(x_t^{(d,k)} - m_t^{(d,k)} \right)^2, \quad (18)$$

and emit a z-score

$$\hat{x}_t^{(d,k)} = \frac{x_t^{(d,k)} - m_t^{(d,k)}}{\sqrt{q_t^{(d,k)} + \epsilon}}. \quad (19)$$

Operationally, we compute $\hat{x}_{i,t}$ using (m_t, q_t) , score window t , and then update (m_{t+1}, q_{t+1}) using $x_{i,t}$. We use separate rates λ_m and λ_v so that the variance adapts more cautiously than the mean under bursty traffic. If a window has no packets, rate features are set to zero and IAT summaries are marked as missing (the evidence drive in Eq. (10) remains defined through other features). We clip z-scores to a fixed magnitude (default 8) to prevent a single spike from dominating. Thresholds are fixed after burn-in. To limit post burn-in drift, online mean and variance updates use a small adaptation rate and are clipped to prevent abrupt shifts. We also evaluated freezing normalisation after burn-in and observed similar qualitative trends, at the

TABLE V
FEATURE CONTRACT USED IN THE MAIN RUNS ($d = 14$, $B = 10$).

Group	Features
Timing/volume	rate, byte_rate, mean_iat, cv_iat
Size/burstiness	mean_size, small_frac, duty_cycle ($B = 10$)
Contention	share, q
DNS	dns_churn, dns_ttl_var, dns_q
TLS	tls_sni_entropy, tls_cert_valid_days

TABLE VI
NOS-GATE PARAMETERS AND DEFAULTS USED IN THE MAIN RUNS (SEE RELEASED CONFIG SNAPSHOT).

Parameter	Default	Meaning
Δt	0.25	window length
k	20	sigmoid slope in $S_{i,t} = \sigma_k(v_{i,t} - \theta)$
θ	1.0	sigmoid threshold
α	1.0	saturation numerator in $f_{\text{sat}}(v) = \frac{\alpha v^2}{1 + \kappa v^2}$
κ	0.75	saturation denominator scale
β	0.10	linear term in v update
γ	0.25	offset in v update
λ	0.35	decay in v update
χ	0.10	stabilisation around v_{rest}
a, b, μ	1, 1, 0.01	recovery build-up and decay (defaults when not overridden)
p	2	evidence norm $\ \hat{x}_{i,t}\ _p$ (L2)
ζ	1.0	evidence scale (default feature scaling)
v_{max}	3.0	optional clipping bound when enabled

cost of slightly higher false positives; we leave systematic adversarial drift handling to future work.

B. Feature contract

The feature stream is deliberately small and restricted to fields visible at the gateway: headers, timestamps, and sizes, plus DNS and TLS handshake metadata when present. From timing and burstiness we include packet rate, simple inter-arrival summaries (mean and coefficient of variation when at least two packets exist), and a low-frequency pacing indicator computed from counts in B fixed micro-bins within the window. From contention we include (i) clique-level rate share and (ii) an interference index derived from the contention weights W used by the *worlds* generator.

For the main results, we use a fixed $d = 14$ feature contract per flow-window; Table V lists the features, and Table VI lists the defaults of NOS used in the main runs. The duty-cycle feature uses $B = 10$ micro-bins within each window.

We densify to a full (flow, window) grid. If a flow has no packets in window t , rate and byte-rate are set to zero; IAT summaries are set to zero when fewer than two packets are present in the window (since IAT is undefined). In runs that include DNS/TLS features (Table V), when DNS/TLS metadata is not present for a particular flow-window, the corresponding DNS/TLS feature values default to zero under the fixed contract. We also report results under a timing+contention-only contract (i.e., without DNS/TLS features) when those fields are not available in a uniform way. The run manifest records the feature contract and discrete settings (including B and the z-score clip magnitude).

C. End-to-end loop and replay-based impact evaluation

The online pipeline is identical in synthetic and corpus modes. The *worlds* generator emits packet traces with flow keys, timestamps, and sizes, together with contention structure and evasion budgets (Table III). We window packets to produce $x_{i,t}$, normalise online to obtain $\hat{x}_{i,t}$, and run NOS-Gate as a per-flow streaming update (Section IV).

All detectors, including baselines, follow the same stand-alone operating protocol. An assumed-benign burn-in segment is used for training (where applicable) and for collecting burn-in scores. Each flow threshold is then set as a high quantile of burn-in scores (Eq. (1)) and frozen for test. Labels are used only for reporting on held-out time.

Queue impact is evaluated by packet-level WFQ replay in two conditions: (i) baseline replay without gating and (ii) replay with gating driven by each method's actionable flags $z_{i,t}$. Replay logs are produced in microseconds; we report queue metrics in milliseconds for readability. Efficiency is reported as microseconds per (flow, window) record, since gateway cost scales primarily with active flows and the window cadence.

We compare NOS-Gate against three lightweight label-free baselines trained on the same burn-in segment and calibrated with the same high-quantile thresholding protocol. KitNET is implemented as an ensemble of single-hidden-layer autoencoders over feature groups of size $\lceil \sqrt{d} \rceil$ (default grouping), with per-group hidden width set to 0.5 of the group size and an aggregator autoencoder with hidden width 8 operating on the vector of per-group reconstruction errors. Autoencoder is a compact one-hidden-layer MLP autoencoder $\text{Linear}(d \rightarrow 32) + \text{ReLU} + \text{Linear}(32 \rightarrow d)$ trained by reconstruction loss. TinyGRU is a one-layer GRU forecaster with hidden size 24 and lookback 8 windows, followed by a linear head that predicts the next normalised feature vector; its anomaly score is the one-step prediction error. All learnable baselines minimise mean squared error (reconstruction for KitNET/autoencoder; one-step prediction for TinyGRU) using Adam (learning rates as in the released configuration), with a fixed small number of epochs. We do not use early stopping in the reported runs; dropout is disabled and weight decay is set to zero unless stated otherwise.

VI. RESULTS

We report results on reproducible worlds containing malicious episodes under the timing-controlled threat model. All methods operate in a stand-alone, label-free mode: per-flow thresholds are set from the burn-in segment using the same high-quantile rule and then frozen for test (Eq. (1)). We report two alert budgets, $\text{FPR} = 1\%$ and $\text{FPR} = 0.1\%$, and we always show the *achieved* benign false-alarm rate on the held-out test segment under that protocol. We report the *achieved* benign false-positive rate induced by the frozen per-flow thresholds on the held-out test segment. Concretely, for each method we compute the fraction of benign flow-windows in the test segment that produce an alarm under $a_{i,t} = \mathbb{I}\{s_{i,t} \geq \vartheta_i\}$, where ϑ_i is set from the burn-in segment (Eq. (1)) and then frozen. Because this estimate aggregates over a very large number of benign flow-windows per world in our evaluation,

small world-to-world differences can fall below the precision implied by typical rounded tables. We therefore state operating points by their target alert budgets and always report the achieved rate under the same protocol. In our harness, achieved FPR is estimated on a benign-only episode over the full horizon, which yields $64 \times 57,600 \approx 3.69 \times 10^6$ flow-windows per world at $\Delta t = 250$ ms. At target FPR = 0.1%, the sampling variability of an empirical false-positive fraction at this n is on the order of 10^{-5} , so values rounded to three decimals can appear identical across worlds. For this reason we state operating points in terms of the target budget and clarify the estimation volume, and we report achieved values at higher precision in the artefact summaries.

A. Incident detection under timing-controlled evasion

Figure 2 summarises incident recall and time-to-detect (TTD) at FPR = 0.1%. Time-to-detect (TTD) is measured from the first malicious-labelled window of an episode to the first threshold-crossing alarm $a_{i,t} = 1$ (i.e., the first window where $s_{i,t} \geq \vartheta_i$ on the attacked flow), and is reported separately from the persistence-based actionable signal used for WFQ gating. This operating point is intentionally strict. Because ϑ_i is a high burn-in quantile and is then frozen, a detector must exceed a tail threshold on each flow. Under timing-controlled evasion (Table III), the attacker can often reduce sharp per-window anomalies by spreading activity across windows while still meeting the throughput floor. This makes methods that depend on occasional high-amplitude windows easier to suppress below ϑ_i , while methods that accumulate modest deviation energy across time are more likely to trigger without relying on a single extreme window.

At FPR = 0.1%, NOS-Gate misses 1 of 21 incidents (incident recall 0.952). The strongest baseline in these runs, TinyGRU, misses 3 incidents (recall 0.857). The remaining baselines miss 5 incidents (recall 0.762).

Figure 2(a) shows per-world recall and indicates that the gain is not driven by a single outlier world. Figure 2(b) shows the TTD distribution over detected incidents. Several baselines can trigger quickly when they trigger, but under the strict budget they fail to trigger on more episodes. NOS-Gate trades some latency on the hardest cases for fewer missed incidents, which is the relevant trade-off when false alarms must remain rare.

Figure 2(c) provides episode-level discordant counts pooled across the same 21 episodes. NOS-Gate detects more episodes that each baseline misses than vice versa (KitNET: 5 vs 1; TinyGRU: 3 vs 1; autoencoder: 5 vs 1). The sample is modest, so we treat this as supporting evidence rather than a standalone statistical claim, but the direction is consistent across baselines.

B. Queue impact of WFQ gating

Detection is evaluated here through its control effect. We therefore assess WFQ gating using packet-level replay, driven by each method's actionable flags under the strict operating point (FPR = 0.1%). Queue logs are produced in microseconds and reported in milliseconds.

Figure 3 reports tail queueing delay and tail collateral delay under gating (p99.9). NOS-Gate is the only method

TABLE VII
SCORING COST ON CPU (MICROSECONDS PER WINDOW-ROW). TIMINGS ARE WALL-CLOCK PER (FLOW, WINDOW) FEATURE ROW ON **INTEL CORE i7-1355U**, USING **WINDOWS 11** AND **[PYTHON 3.14.2]**; MEASUREMENTS WERE TAKEN IN **[SINGLE-THREAD]** MODE. REPORTED TIME INCLUDES ONLINE NORMALISATION AND MODEL UPDATE/SCORING GIVEN PRECOMPUTED WINDOW FEATURES; IT EXCLUDES PACKET CAPTURE/PARSING AND WINDOW FEATURE EXTRACTION.

Method	Mean (μ s/row)	p90 (μ s/row)	Max (μ s/row)
NOS-Gate	2.09	2.97	2.99
Autoencoder	2.03	2.78	2.80
KitNET	11.68	16.72	16.80
TinyGRU	31.10	46.57	49.24

that improves both metrics consistently across the 7 worlds. Relative to baseline replay without gating, NOS-driven gating reduces p99.9 queueing delay by about 3.24 ms on average across worlds and reduces p99.9 collateral delay by about 3.16 ms. In contrast, gating driven by KitNET and the autoencoder increases mean p99.9 queueing delay in these runs (about +0.30 ms and +0.91 ms, respectively), which is consistent with less targeted gating under the same strict alert budget.

Figure 3(c) summarises the central deployment question: under a strict alert cap, does the detector sit in the favourable region of higher incident recall and lower tail delay. Under the shared stand-alone protocol, NOS-Gate occupies that region in these worlds.

C. Efficiency on CPU

Table VII reports scoring cost in microseconds per window-row. NOS-Gate remains lightweight (mean $\approx 2.09 \mu$ s/row), comparable to the compact autoencoder and substantially faster than KitNET and TinyGRU in these runs. This matters because gateway cost scales with active flows and window cadence. A method that is expensive per window is harder to sustain without adding latency. All methods are timed under the same measurement protocol; we report per-row cost because gateway compute scales primarily with active flows and the fixed window cadence.

D. Summary

Across these runs, NOS-Gate is the only method that meets the three constraints that define the gateway setting at once: (i) under a strict false-alarm budget it misses fewer incidents, (ii) its actionable flags drive a reversible WFQ action that reduces both queue-tail delay and collateral tail delay in replay, and (iii) it achieves this with a small, stable per-window compute cost. Under looser thresholds, baselines can appear closer. The strict stand-alone calibration exposes the failure mode that matters operationally: missed incidents and action-triggered latency when alerts must be rare.

VII. CONCLUSION

Consumer gateways in the networks sit at the point where encrypted services meet heterogeneous IoT devices, but encryption leaves timing, size, and burst structure visible. Those side channels support metadata-based IDS, yet they

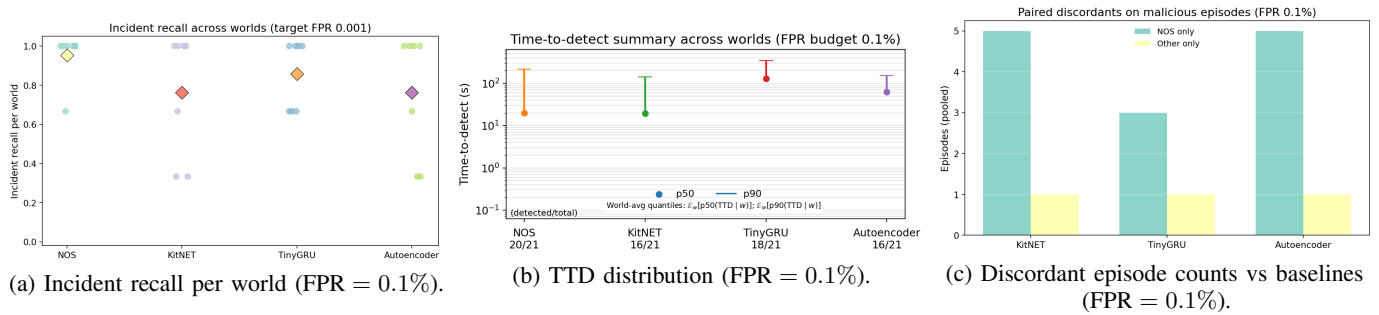


Fig. 2. Detection under the strict false-alarm budget (TTD is defined to the first alarm $a_{i,t} = 1$, not to the first actionable trigger under K -of- M persistence.). Panels (a) and (b) show per-world behaviour and latency dispersion, not only pooled totals. Panel (c) summarises episode-level discordants (NOS-Gate only vs baseline), pooled across worlds.

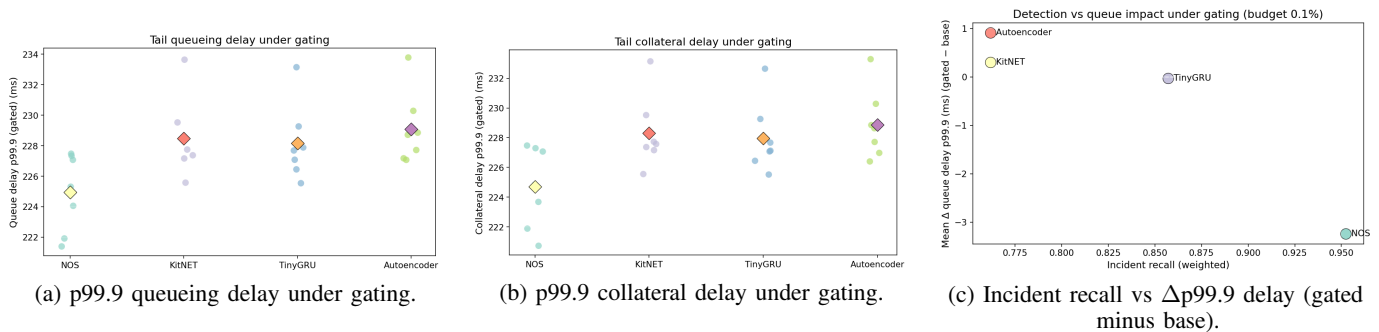


Fig. 3. Action-level evaluation under the strict operating point (FPR = 0.1%). NOS-Gate is the only method that improves both queue-tail delay and collateral tail delay while maintaining the highest incident recall under the same label-free calibration protocol. Delays are reported in ms (raw logs are in μ s).

also create a practical evasion surface: an adaptive adversary can reshape timing while still meeting an attack objective. This paper addressed a deployment-centred question: under a strict false-alarm cap and without labels, can a gateway detect quickly enough to act, using only metadata, and can that action reduce user-visible queue harm rather than add to it? We introduced *NOS-Gate*, a streaming IDS that instantiates a lightweight two-state unit derived from Network-Optimised Spiking (NOS) dynamics per flow. The model is structurally faithful to NOS and changes only the interpretation of the exogenous drive, which becomes a scalar evidence signal derived from online normalised metadata features. The resulting state update provides controlled leak, bounded growth, and recovery-driven suppression, producing a stable score that is calibrated stand-alone by burn-in quantile thresholding and stabilised by a K -of- M persistence rule. Crucially, NOS-Gate connects detection to an auditable mitigation path by temporarily reducing WFQ weights for persistently suspicious flows. To evaluate this action loop under timing-controlled evasion, we introduced an executable *worlds* benchmark with explicit attacker budgets and packet-level WFQ replay. Under the strict operating point (0.1% achieved benign false-positive rate), NOS-Gate achieved the highest incident recall among the tested label-free baselines while also improving tail queue outcomes under gating, including reduced p99.9 queueing delay and reduced collateral tail delay. Its per-window scoring cost remained small and stable on CPU, which matches gateway scaling constraints. Finally, we do not claim payload-level coverage or that synthetic worlds capture

every real deployment. Instead, we offer a falsifiable protocol, artefacts, and budgets that make both evasion and queue impact measurable. NOS-Gate operates per flow with a persistence rule, which is appropriate when malicious activity is sustained on a small number of flows. An adaptive adversary could attempt to dilute evidence by increasing flow churn (many short-lived flows) or by spreading activity across multiple endpoints so that no single flow accumulates enough alarms within the persistence window to trigger action. We do not evaluate such multi-flow strategies in this paper; addressing them may require lightweight aggregation above the five-tuple (for example, per-device or per-destination persistence) or coupling across flows sharing a scheduling domain, while preserving the same stand-alone calibration and auditable action constraints. Future work should expand the benchmark with additional device processes, and explore how learned feature representations can be introduced without breaking the stand-alone calibration and action-audit requirements.

REFERENCES

- [1] M. Bilal, "Network-optimised spiking neural network for event-driven networking," *arXiv preprint arXiv:2509.23516*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.23516>
- [2] M. Conti, L. V. Mancini, R. Spolaor, and N. V. Verde, "Analyzing android encrypted network traffic to identify user actions," *IEEE Transactions on Information Forensics and Security*, vol. 11, no. 1, pp. 114–125, 2016.
- [3] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle, "Website fingerprinting at internet scale," in *Network and Distributed System Security Symposium*, 2016. [Online]. Available: <https://api.semanticscholar.org/CorpusID:15302617>

- [4] A. Sivanathan, H. H. Gharakheili, F. Loi, A. Radford, C. Wijenayake, A. Vishwanath, and V. Sivaraman, "Classifying iot devices in smart environments using network traffic characteristics," *IEEE Transactions on Mobile Computing*, vol. 18, no. 8, pp. 1745–1759, 2019.
- [5] M. Miettinen, S. Marchal, I. Hafeez, N. Asokan, A.-R. Sadeghi, and S. Tarkoma, "Iot sentinel: Automated device-type identification for security enforcement in iot," in *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*, 2017, pp. 2177–2184.
- [6] I. Hafeez, M. Antikainen, A. Y. Ding, and S. Tarkoma, "Iot-keeper: Detecting malicious iot network activity using online traffic analysis at the edge," *IEEE Transactions on Network and Service Management*, vol. 17, no. 1, pp. 45–59, 2020.
- [7] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A.-R. Sadeghi, "Dfot: A federated self-learning anomaly detection system for iot," in *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*, 2019, pp. 756–767.
- [8] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," vol. 41, no. 3, 2009.
- [9] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Network and Distributed System Security Symposium (NDSS)*, 2018.
- [10] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic morphing: An efficient defense against statistical traffic analysis," in *Network and Distributed System Security Symposium (NDSS)*, 2009.
- [11] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A systematic approach to developing and evaluating website fingerprinting defenses." New York, NY, USA: Association for Computing Machinery, 2014.
- [12] M. Bilal and X. Xu, "Network-optimised spiking neural network (nos) scheduling for 6g o-ran: Spectral margin and delay-tail control," *arXiv preprint arXiv:2510.11291*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.11291>
- [13] A. Tavanaei, M. Ghodrati, S. R. Kheradpisheh, T. Masquelier, and A. Maida, "Deep learning in spiking neural networks," *Neural Networks*, vol. 111, pp. 47–63, 2019.
- [14] D. Marković, A. Mizrahi, D. Querlioz, and J. Grollier, "Physics for neuromorphic computing," *Nature Reviews Physics*, 2020.
- [15] A. Demers, S. Keshav, and S. Shenker, "Analysis and simulation of a fair queueing algorithm," *SIGCOMM Comput. Commun. Rev.*, vol. 19, no. 4, p. 1–12, 1989. [Online]. Available: <https://doi.org/10.1145/75247.75248>
- [16] A. Parekh and R. Gallager, "A generalized processor sharing approach to flow control in integrated services networks: the single-node case," *IEEE/ACM Transactions on Networking*, vol. 1, no. 3, pp. 344–357, 1993.
- [17] A. K. Parekh and R. G. Gallager, "A generalized processor sharing approach to flow control in integrated services networks: The multiple-node case," *IEEE/ACM Transactions on Networking*, vol. 2, no. 2, 1994.
- [18] K. Nichols and V. Jacobson, "Controlling queue delay," *Queue*, vol. 10, no. 5, 2012.
- [19] C. Fu, Q. Li, and K. Xu, "Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis," in *Network and Distributed System Security Symposium (NDSS)*, 2023.
- [20] Y. Qing, Q. Yin, X. Deng, Y. Chen, Z. Liu, K. Sun, K. Xu, J. Zhang, and Q. Li, "Low-quality training data only? A robust framework for detecting encrypted malicious network traffic," in *Network and Distributed System Security Symposium (NDSS)*, 2024.
- [21] L. Gao, C. Fu, X. Deng, K. Xu, and Q. Li, "Wedjat: Detecting sophisticated evasion attacks via real-time causal analysis," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, ser. KDD '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 342–353. [Online]. Available: <https://doi.org/10.1145/3690624.3709218>
- [22] S. Sattar, S. Khan, M. I. Khan, A. Akhmediyarova, O. Mamyrbayev, D. Kassymova, D. Oralbekova, and J. Alimkulova, "Anomaly detection in encrypted network traffic using self-supervised learning," *Scientific Reports*, vol. 15, p. 26585, 2025. [Online]. Available: <https://www.nature.com/articles/s41598-025-08568-0>
- [23] Y. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, A. Shabtai, D. Breitenbacher, and Y. Elovici, "N-baiot—network-based detection of iot botnet attacks using deep autoencoders," *IEEE Pervasive Computing*, vol. 17, no. 3, pp. 12–22, 2018.
- [24] S. Sicari, A. Rizzardi, L. Grieco, and A. Coen-Porisini, "Security, privacy and trust in internet of things: The road ahead," *Computer Networks*, vol. 76, pp. 146–164, 2015. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1389128614003971>
- [25] H. Tahaei, F. Afifi, A. Asemi, F. Zaki, and N. B. Anuar, "The rise of traffic classification in iot networks: A survey," *Journal of Network and Computer Applications*, vol. 154, p. 102538, 2020.
- [26] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.